

Why Refactor

.Dozens of checks for null make your code longer and uglier

Drawbacks

.The price of getting rid of conditionals is creating yet another new class

How to Refactor

From the class in question, create a subclass that will perform the role of
.null object

In both classes, create the method `isNull()`, which will return true for a
.null object and false for a real class

Find all places where the code may return null instead of a real object.
.Change the code so that it returns a null object

Find all places where the variables of the real class are compared with
.()null. Replace these checks with a call for `isNull`

If methods of the original class are run in these conditionals when a
value doesn't equal null, redefine these methods in the null class and
insert the code from the else part of the condition there. Then you can
delete the entire conditional and differing behavior will be implemented
.via polymorphism

If things aren't so simple and the methods can't be redefined, see if you
can simply extract the operators that were supposed to be performed in
the case of a null value to new methods of the null object. Call these
.methods instead of the old code in else as the operations by default

Introduce Null Object

مشکل (Problem)

بعضی متدها به جای شیء واقعی، null برمی‌گردونن و این باعث میشه کلی چک null توی کدت
پخش بشه.

راه‌حل (Solution)

به جای `null` ، به **Null Object** برگردون که رفتار پیش فرض رو پیاده سازی می کنه.

مثال

قبل از ریفکتور:

csharp

```
if (customer == null)
{
    ;()plan = BillingPlan.Basic
}
else
{
    ;()plan = customer.GetPlan
}
```

بعد از Introduce Null Object:

csharp

```
// کلاس Null که رفتار پیش فرض داره
public sealed class NullCustomer : Customer
{
    ;public override bool IsNull => true

    ()public override Plan GetPlan
    {
        ;()return new NullPlan
    }
}

// جایگزینی با null با Null-object
;()customer = order.customer ?? new NullCustomer

// دیگه نیاز به چک null نیست!
;()plan = customer.GetPlan
```

چرا ریفکتور کنیم؟ (Why Refactor)

ده‌ها چک `null` کد رو طولانی‌تر و زشت‌تر می‌کنه.

نقطه ضعف (Drawbacks)

قیمت خلاص شدن از شرط‌ها، ساختن یه کلاس جدید دیگه‌ست.

How to Refactor (چطور ریفکتور کنیم)

توضیح	قدم
یه زیرکلاس از کلاس مورد نظر بساز که نقش <code>Null Object</code> رو بازی کنه	۱
توی هر دو کلاس، متد <code>IsNull</code> رو بساز (برای <code>Null Object</code> مقدار <code>true</code> (	۲
جاهایی که <code>null</code> برمی‌گردونن رو پیدا کن و به جاش <code>Null Object</code> برگردون	۳
جاهایی که با <code>null</code> مقایسه می‌کنن رو با <code>IsNull</code> جایگزین کن	۴
اگه متدهای کلاس اصلی توی شرط <code>else</code> (وقتی <code>null</code> نیست) اجرا میشن، این متدها رو توی کلاس <code>Null</code> بازتعریف کن و کد <code>else</code> رو بذار توش. بعد کل شرط رو حذف کن و رفتار متفاوت با <code>Polymorphism</code> انجام همیشه	۵
اگه ساده نیست و متدها قابل بازتعریف نیستن، عملیات مربوط به <code>null</code> رو به متدهای جدید توی <code>Null Object</code> منتقل کن	۶

تمرین

این کد رو با `Introduce Null Object` ریفکتور کن:

```

        public class User
        {
            public string Name { get; set; }
            public string Email { get; set; }

            public virtual string GetDisplayName
            {
                get
                {
                    ;return Name
                }
            }

            public virtual bool IsNull => false
        }

        public class UserService
        {
            public User GetUserById(int id)
            {
                // شبیه سازی - اگر id منفی باشد کاربر وجود ندارد
                if (id < 0)
                {
                    ;return null
                }

                ;{ "Email = "user@test.com", "کاربر واقعی" = Name } return new User
            }

            public void ShowUserInfo(int id)
            {
                ;var user = GetUserById(id)

                // چک null توی همه جا پخش شده
                ;string displayName
                if (user == null)
                {
                    ;"کاربر مهمان" = displayName
                }
                else
                {
                    ;()displayName = user.GetDisplayName
                }

                ;string email
                if (user == null)
                {

```

```

        ; ایمیل نامشخص" = email
        {
            else
            }
        ;email = user.Email
        {

;("{email} , ایمیل: {displayName} نام:$")Console.WriteLine
        {
            {

```

راهنمایی:

- به کلاس User : NullUser بساز
- IsNull رو true کن
- GetDisplayName رو override کن که "کاربر مهمان" برگردونه
- GetUserById رو طوری تغییر بده که NullUser برگردونه به جای null
- شرطهای null رو حذف کن

بفرست تا بررسی کنیم! 📩